Operators and Expressions in C

Theory and Worked Programs for Each Operator Category

Topics Covered
1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Unary Operators
6. Increment / Decrement Operators
7. Conditional (Ternary) Operator

Scope note

Every program in this guide uses only variables, data types, and `scanf`/`printf` alongside the operator being taught. `if`/`else`, loops, functions, and arrays are intentionally not used, since those topics come later in the course. Relational and logical results are simply printed as **1 (true)** or **0 (false)** — exactly how C represents them — rather than being fed into a branching statement.

1. Arithmetic Operators

Arithmetic operators perform basic mathematical calculations on numeric operands.

Operator	Meaning	Example (a=10, b=3)	Result
+	Addition	a + b	13
-	Subtraction	a - b	7
*	Multiplication	a * b	30
/	Division	a / b	3 (integer division discards the decimal part)
%	Modulus (remainder)	a % b	1 — only works on integers, never on float/double

Program: arithmetic calculator

```
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);

    printf("Sum        = %d\n", a + b);
    printf("Difference = %d\n", a - b);
    printf("Product    = %d\n", a * b);
    printf("Quotient   = %d\n", a / b);
    printf("Remainder  = %d\n", a % b);

    return 0;
}
```

```
$ gcc program1.c -o program1
$ ./program1
Enter first number: 10
Enter second number: 3
Sum        = 13
Difference = 7
Product    = 30
Quotient   = 3
Remainder  = 1
```

Common mistake

`%` only works with `int` operands. `10.0 % 3.0` will not compile — it produces a compilation error, not a wrong answer, so it's caught immediately.

2. Relational Operators

Relational operators compare two values and produce a result of **1** (true) or **0** (false). C has no separate boolean type — these results are ordinary integers.

Operator	Meaning	Example (a=10, b=3)	Result
<	Less than	a < b	0
>	Greater than	a > b	1
<=	Less than or equal to	a <= b	0
>=	Greater than or equal to	a >= b	1
==	Equal to	a == b	0
!=	Not equal to	a != b	1

Program: comparing two numbers

```
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);

    printf("a < b  : %d\n", a < b);
    printf("a > b  : %d\n", a > b);
    printf("a <= b : %d\n", a <= b);
    printf("a >= b : %d\n", a >= b);
    printf("a == b : %d\n", a == b);
    printf("a != b : %d\n", a != b);

    return 0;
}
```

```
$ ./program2
Enter first number: 10
Enter second number: 3
a < b  : 0
a > b  : 1
a <= b : 0
a >= b : 1
a == b : 0
a != b : 1
```

Common mistake

Writing `a = b` (single =) when you meant `a == b` (double =) is one of the most frequent bugs in C. The first one *assigns* b's value to a; the second one *compares* them. This will matter a great deal once you reach `if` statements.

3. Logical Operators

Logical operators combine or invert true/false (1/0) values, usually built from relational expressions.

Operator	Meaning	Rule
&&	Logical AND	True only if BOTH operands are true (non-zero)
||	Logical OR	True if AT LEAST ONE operand is true (non-zero)
!	Logical NOT	Reverses the value: true becomes false, and vice versa

Program: checking eligibility conditions

```
#include <stdio.h>

int main() {
    int age, marks;

    printf("Enter age: ");
    scanf("%d", &age);
    printf("Enter marks (out of 100): ");
    scanf("%d", &marks);

    printf("age >= 18 AND marks >= 50 : %d\n", (age >= 18) && (marks >= 50));
    printf("age >= 18 OR  marks >= 50 : %d\n", (age >= 18) || (marks >= 50));
    printf("NOT (age >= 18)           : %d\n", !(age >= 18));

    return 0;
}
```

```
$ ./program3
Enter age: 20
Enter marks (out of 100): 40
age >= 18 AND marks >= 50 : 0
age >= 18 OR  marks >= 50 : 1
NOT (age >= 18)           : 0
```

Here age (20) satisfies age ≥ 18 but marks (40) fails marks ≥ 50. AND needs both conditions, so it prints 0; OR only needs one, so it prints 1.

4. Assignment Operators

Beyond the simple = you already use, C provides **compound assignment operators** that combine an arithmetic operation with assignment in one step.

Operator	Meaning	Equivalent to	Example (x=10)
=	Simple assignment	—	x = 10
+=	Add and assign	x = x + value	x += 5 → x = 15
-=	Subtract and assign	x = x - value	x -= 5 → x = 5
*=	Multiply and assign	x = x * value	x *= 5 → x = 50
/=	Divide and assign	x = x / value	x /= 5 → x = 2
%=	Modulus and assign	x = x % value	x %= 3 → x = 1

Program: compound assignment step by step

```
#include <stdio.h>

int main() {
    int x = 10;
    printf("Initial value: x = %d\n", x);

    x += 5;
    printf("After x += 5 : x = %d\n", x);

    x -= 3;
    printf("After x -= 3 : x = %d\n", x);

    x *= 2;
    printf("After x *= 2 : x = %d\n", x);

    x /= 4;
    printf("After x /= 4 : x = %d\n", x);

    x %= 5;
    printf("After x %%= 5: x = %d\n", x);

    return 0;
}
```

```
$ ./program4
Initial value: x = 10
After x += 5 : x = 15
After x -= 3 : x = 12
After x *= 2 : x = 24
After x /= 4 : x = 6
After x %= 5: x = 1
```

Why use compound assignment

`x += 5` says the same thing as `x = x + 5` but is shorter and avoids typing the variable name twice — useful once totals start building up across many steps.

5. Unary Operators

A unary operator acts on a **single** operand (compare this with arithmetic operators like +, which need two). The two you'll use most often are unary + and unary -.

Operator	Meaning	Example	Result
-	Unary minus — negates a value	-a (a=5)	-5
+	Unary plus — has no real effect, sign is already positive by default	+a (a=5)	5

Program: unary minus and unary plus

```
#include <stdio.h>

int main() {
    int a;

    printf("Enter a number: ");
    scanf("%d", &a);

    printf("Original value : %d\n", a);
    printf("Unary minus (-a): %d\n", -a);
    printf("Unary plus  (+a): %d\n", +a);

    return 0;
}
```

```
$ ./program5
Enter a number: 7
Original value : 7
Unary minus (-a): -7
Unary plus  (+a): 7
```

Note the difference from binary subtraction: a - b needs two operands, while -a needs only one — it simply flips the sign of whatever value a holds.

6. Increment / Decrement Operators

++ increases a variable's value by 1, and `--` decreases it by 1. Each comes in two forms that behave differently *inside an expression*, even though they leave the same final value in the variable.

Form	Name	Behavior in an expression
++a	Pre-increment	Increases a FIRST, then uses the new value in the expression
a++	Post-increment	Uses the CURRENT value in the expression first, then increases a
--a	Pre-decrement	Decreases a FIRST, then uses the new value in the expression
a--	Post-decrement	Uses the CURRENT value in the expression first, then decreases a

Program: pre vs. post increment/decrement

```
#include <stdio.h>

int main() {
    int a = 5, b = 5;
    int resultPre, resultPost;

    resultPre  = ++a;     // a becomes 6 first, THEN resultPre gets 6
    resultPost = b++;     // resultPost gets 5 FIRST, then b becomes 6

    printf("Pre-increment:  a = %d, resultPre  = %d\n", a, resultPre);
    printf("Post-increment: b = %d, resultPost = %d\n", b, resultPost);

    int c = 5, d = 5;
    int resultPreDec, resultPostDec;

    resultPreDec  = --c;
    resultPostDec = d--;

    printf("Pre-decrement:  c = %d, resultPreDec  = %d\n", c, resultPreDec);
    printf("Post-decrement: d = %d, resultPostDec = %d\n", d, resultPostDec);

    return 0;
}
```

```
$ ./program6
Pre-increment:  a = 6, resultPre  = 6
Post-increment: b = 6, resultPost = 5
Pre-decrement:  c = 4, resultPreDec  = 4
Post-decrement: d = 4, resultPostDec = 5
```

Why this trips up beginners

In both cases the variable itself ends up at the same final value (6 or 4). The difference only shows up in **what gets copied into the result variable** — the value before, or the value after, the change. This distinction becomes very important once you start using these operators inside loop conditions.

7. Conditional (Ternary) Operator

The conditional operator `?:` is C's only operator that takes **three** operands. It is a compact way to pick one of two values based on a condition, written as:

```
condition ? valueIfTrue : valueIfFalse
```

If `condition` evaluates to true (non-zero), the whole expression takes the value of `valueIfTrue`; otherwise it takes `valueIfFalse`.

Program: finding the larger of two numbers

```
#include <stdio.h>

int main() {
    int a, b, larger;

    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);

    larger = (a > b) ? a : b;

    printf("The larger number is: %d\n", larger);

    return 0;
}
```

```
$ ./program7
Enter first number: 15
Enter second number: 42
The larger number is: 42
```

Here `(a > b)` is false (15 is not greater than 42), so the whole expression evaluates to `b`, which is 42.

Relating this to if/else

`larger = (a > b) ? a : b;` does exactly what a full `if/else` statement would do, just in a single line. You'll see the longer `if/else` form in the next topic — the ternary operator is simply a shorthand for the same idea when there are only two possible outcomes.

Summary: Operator Precedence Preview

When several operators appear in the same expression, C evaluates them in a fixed order of precedence — roughly the same "multiplication before addition" rule you already know from mathematics, extended to cover every operator in this guide.

Precedence (high → low)	Operators	Example
1. Unary	! , unary + , unary - , ++ , --	-a, !flag, ++count
2. Arithmetic	* , / , % then + , -	a + b * c → multiplication happens first
3. Relational	< , <= , > , >= then == , !=	a + 1 > b → addition happens first
4. Logical AND	&&	a > 0 && b > 0
5. Logical OR	||	a > 0 || b > 0
6. Conditional	?:	(a > b) ? a : b
7. Assignment	= , += , -= , *= , /= , %=	x += 5

Practical rule of thumb

When in doubt, add parentheses. `(a > 0) && (b > 0)` is exactly as correct as `a > 0 && b > 0`, but far easier for a reader (including you, a week later) to check at a glance — this connects back to the code readability practices you've already covered.

All programs use only variables, data types, and scanf/printf alongside each operator category. if/else, loops, functions, and arrays are deliberately not used, since they are introduced in later lectures.